新闻文本分类，使用长短期记忆网络（LSTM）来完成，长短期记忆网络是循环神经网络的变体，建议从最普通的循环神经网络开始。

1.将字符串表示转成向量表示

将字符串拆分建立词表，然后将句子的字符串表示变成由句子单词 ID 表示的向量。

2.创建模型并训练

创建 LSTM 模型，本文使用 pytorch，主要包含几块内容：词表示，网络前馈计算，将结果输出。在 pytorch 中所有的计算过程都在 forward 中实现，不需要手写，不过建议首先手动推导整个计算过程，然后对应到 pytorch 框架上进行实现。训练过程中使用交叉熵损失进行参数优化，pytorch 中只需要执行一下 backward()方法就能实现误差你传播，梯度计算。

3.执行预测生成结果

在训练完成之后，使用训练结果进行数据预测，预测方法也很简单，和训练过程基本类似，由于本文中我们使用的是 one-hot 表示，因此，输出向量大小为 14.执行完预测之后，按照赛题给出的测试数据将预测结果写入 csv 文件。

导入包

```
import torch
import torch.nn as nn
import torchtext
from torchtext.data.utils import get_tokenizer
from torchtext.vocab import Vocab
from collections import Counter
from torchtext.datasets import AG_NEWS
import pandas as pd

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

获取训练集和测试集

```
# 训练数据
dataset = pd.read_csv("data/train_set.csv", sep="\t")
# 训练集
train_dataset = dataset.sample(frac=0.9)

# 测试集
test_dataset = dataset[~dataset.index.isin(train_dataset.index)]
```

```
# 创建次表
tokenizer = get_tokenizer("basic_english")
counter = Counter()
for (lebel, line) in train_dataset.iloc:
    counter.update(tokenizer(line))

# 词表
vocab = Vocab(counter, min_freq=1)

# 单词总数
vocab_num = len(vocab)
```

```
# 将英文句子转成ID列表
text_pipeline = lambda x: [vocab[token] for token in tokenizer(x)]

# 将标签转成ID
label_pipeline = lambda x: int(x)
```

获取句子向量

```
# 将训练集句子转成ID向量
def get_label_line_tensors(data):
    lines = []
    labels = []
    for (label, line) in data:
        lines.append(torch.tensor(text_pipeline(line)).to(device))
        labels.append(torch.tensor([label_pipeline(label)]).to(device))
    return labels, lines


labels, lines = get_label_line_tensors(train_dataset.iloc)
```

定义模型

```
# 定义模型
class RNN_LSTM(nn.Module):
    def __init__(self, input_size, hidden_size, output_size,  n_layers, vocab
        super(RNN_LSTM, self).__init__()
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.n_layers = n_layers
        self.vocab_size = vocab_size
        self.dropout_p = dropout_p

        # 定义神经网络层
        self.embeding = nn.Embedding(vocab_size, input_size)
        self.rnn = nn.LSTM(input_size, hidden_size, n_layers, dropout=self.dr
        self.out = nn.Linear(hidden_size, output_size)
        self.softmax = nn.LogSoftmax(dim=0)

    # 前馈
    def forward(self, input_words, hidden, cell):
        seq_len = input_words.size()[0]
        embeding = self.embeding(input_words).view(seq_len, 1, -1)
        out, (hn, cn) = self.rnn(embeding, (hidden, cell))
```

```
        out = self.softmax(self.out(out))
        return out, (hn, cn)

    # 初始化隐含层
    def init_hidden(self):
        hidden = torch.zeros(self.n_layers, 1, self.hidden_size)
        return hidden.to(device)

    # 初始化 cell state
    def init_cell_state(self):
        cell = torch.zeros(self.n_layers, 1, self.hidden_size)
        return cell.to(device)
```

```
#定义模型
model = RNN_LSTM(128, 128, 14, 2, vocab_num, 0.1).to(device)
# 数据训练
def train(line, label, loss):
    optimizer.zero_grad()
    hidden = model.init_hidden()
    cell = model.init_cell_state()
    o, (h, c)  = model(line, hidden, cell)
    #oh = torch.sum(o, dim=0)
    oh = o[-1:].squeeze(0)
```

```
    l = loss(oh, label)

    l.backward()
    optimizer.step()
    return l

#损失函数
loss = nn.CrossEntropyLoss()
import random
import time

#执行训练
lossNum = []
begin = time.time()
print(begin)
optimizer = torch.optim.Adam(model.parameters(), lr=5e-4)
for i in range(600001):
    idx = random.randint(0, 179999)
    l = train(lines[idx], labels[idx], loss)
    lossNum.append(l)
    if i % 10000==0 or i==(159999):
        print(l)
print(time.time()-begin)
```

```
t_labels, t_lines = get_label_line_tensors(test_dataset.iloc)
# 测试数据预测
check_h = model.init_hidden()
check_c = model.init_cell_state()
check_o, (_, _) = model(t_lines[0], check_h, check_c)
# check_o 即为预测向量
```

至此，赛题主要代码就完成了，预测时只需要选择输入结果向量中值最大的元素索引作为标签ID。